

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / C Ruiz / D García / S villanova

Primer Semestre 2026

Universidad Andrés Bello

Departamento de Física y Astronomía



Resumen - Semana 8, Sesión 2 (Sesión 16)

Introducción y Conexión

Tema y Dataset

Ejercicios Prácticos Integrados

Cierre y Retroalimentación

Conclusiones

Introducción y Conexión

Introducción y Conexión

Repaso de la Sesión Previa

- **Semana 8, Sesión 1 (P1):**
 - Fundamentos de **pandas**: Series, DataFrames, selección, filtrado, groupby.
 - Limpieza de datos (NaN), estadísticas y visualización con Matplotlib.
- **Objetivo de hoy:** Aplicar estos conceptos cargando un archivo CSV real y practicando herramientas básicas de pandas.

Introducción y Conexión

Objetivos de la Sesión P2

- **Consolidar** lectura de archivos con `pd.read_csv()`.
- **Practicar** acceso, selección y filtrado en DataFrames.
- **Trabajar** con datos faltantes usando `isnull()`, `dropna()` y `drop()`.
- **Integrar** agrupación, visualización simple y exportación a CSV.

Tema y Dataset

Contexto

- Trabajaremos con un archivo llamado `'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/planets.csv'`.
- La idea es practicar un flujo simple y realista de trabajo con pandas.
- El dataset puede contener columnas numéricas y categóricas, por ejemplo:
 - una columna identificadora
 - mediciones físicas
 - alguna categoría o tipo de objeto
 - posibles valores faltantes

Meta: Lectura → inspección → selección → filtrado → limpieza → agrupación → visualización → exportación.

Tema y Dataset

Lectura del dataset:

Archivo **experimento.csv**



```
1 import pandas as pd
2
3 df = pd.read_csv("https://raw.githubusercontent.com/mwaskom/seaborn-data/master/planets.csv")
4
5 df.head()
```

Discusión: En esta sesión nos interesa trabajar con datos cargados desde archivo, no generarlos artificialmente con código.

Ejercicios Prácticos Integrados



Enunciado

- Lee el archivo con
`pd.read_csv("https://raw.githubusercontent.com/mwasko/data/master/planets.csv")`.
- Muestra `df.head()`, `df.shape`, `df.columns` y `df.info()`.

Conceptos: Lectura de CSV e inspección inicial.



```
1 import pandas as pd
2
3 df = pd.read_csv("https://raw.githubusercontent.com/mwaskom/seaborn-data/master/planets.csv")
4
5 print(df.head())
6 print("shape:", df.shape)
7 print("columnas:", df.columns.tolist())
8 print("="*40)
9 df.info()
```

Discusión: Antes de analizar cualquier dataset conviene ver su estructura general y sus columnas disponibles.



Enunciado

- Selecciona un subconjunto de columnas relevantes.
- Muestra `df.describe()` para las columnas numéricas del dataset.
- Indica cuáles columnas son numéricas y cuáles no.

Objetivo: Reconocer tipos de columnas y obtener un resumen estadístico básico.



```
1 subset = df.iloc[:, :4]
2 print(subset.head())
3
4 print(df.describe())
5
6 num_cols = df.select_dtypes(include='number').columns.tolist()
7 non_num_cols = df.select_dtypes(exclude='number').columns.tolist()
8 print("Numéricas:", num_cols)
9 print("No numéricas:", non_num_cols)
```

Discusión: Distinguir tipos de columnas ayuda a decidir qué operaciones tienen sentido sobre cada variable.



Enunciado

- Filtra el DataFrame con una condición simple sobre una columna numérica.
- Luego aplica un filtrado múltiple con dos o tres condiciones usando `&`.
- Muestra el subconjunto resultante.

Conceptos: Máscaras booleanas y selección.

Solución 3 de Referencia: Filtrado simple y múltiple



```
1 filtro_simple = df[df['temperatura'] > 300]
2 print(filtro_simple.head())
3
4 filtro_multiple = df[
5     (df['temperatura'] > 300) &
6     (df['masa'] > 1.0)
7 ]
8 print(filtro_multiple)
```

Discusión: El filtrado es una de las operaciones más frecuentes cuando se trabaja con tablas de datos.



Enunciado

- Cuenta valores faltantes por columna con `isnull()`.
- Crea un DataFrame limpio con `dropna()`.
- Compara cuántas filas había antes y después de limpiar.

Objetivo: Identificar y eliminar filas incompletas.



```
1 print(df.isnull().sum())
2
3 df_clean = df.dropna()
4
5 print("Filas originales:", len(df))
6 print("Filas sin nulos:", len(df_clean))
```

Discusión: Primero se detectan los nulos y luego se decide si conviene eliminarlos o tratarlos de otra forma.



Enunciado

- Elimina una o más columnas que no sean necesarias para el análisis inmediato.
- Guarda el resultado en un nuevo DataFrame.
- Muestra las columnas que quedaron.

Conceptos: Uso de `drop(columns=[...])`.



```
1 df_reducido = df.drop(columns=['comentario'])  
2 print(df_reducido.columns.tolist())
```

Discusión: Quitar columnas irrelevantes deja un DataFrame más claro y manejable para el análisis.



Enunciado

- Agrupa el DataFrame por una columna categórica.
- Calcula la media de una o dos columnas numéricas para cada grupo.
- Muestra el resultado.

Objetivo: Resumir información por grupos usando pandas.



```
1 resumen = df.groupby('categoria')[['temperatura', 'masa']].mean()  
2 print(resumen)
```

Discusión: `groupby( ).mean( )` permite resumir un dataset de forma compacta y muy útil.



Enunciado

- Dibuja un histograma simple de una columna numérica.
- Crea un gráfico de barras con conteos por categoría.
- Exporta un DataFrame limpio o filtrado a un archivo CSV.

Objetivo: Integrar visualización rápida y persistencia de resultados.



```
1 import matplotlib.pyplot as plt
2
3 df['temperatura'].hist(bins=20)
4 plt.title('Histograma de temperatura')
5 plt.xlabel('Temperatura')
6 plt.ylabel('Frecuencia')
7 plt.show()
8
9 df['categoria'].value_counts().plot(kind='bar')
10 plt.title('Conteo por categoría')
11 plt.ylabel('Cantidad')
12 plt.show()
13
14 df_clean.to_csv('experimento_limpio.csv', index=False)
```

Discusión: Se combinan dos tareas frecuentes del análisis exploratorio: visualizar rápidamente y guardar resultados.

Cierre y Retroalimentación

- **Estructuras y conceptos integrados:**
 - pandas: lectura de CSV, inspección, filtrado, limpieza, agrupación, exportación.
 - Visualización: histogramas y barras.
- **Buenas prácticas observadas:**
 - Explorar el DataFrame antes de modificarlo.
 - Identificar nulos antes de aplicar limpieza.
 - Trabajar con subconjuntos y columnas relevantes.

Cierre y Retroalimentación

Retroalimentación Colectiva

- ¿Qué comando les resultó más útil para inspeccionar el dataset?
- ¿Qué diferencia hay entre `drop()` y `dropna()`?
- ¿En qué casos conviene usar `groupby()`?

Importante

La prioridad en esta etapa es dominar herramientas básicas de pandas sobre datos reales cargados desde archivo.

Conclusiones

- **Consolidamos:** lectura y exploración de archivos CSV con pandas.
- **Aplicamos:** selección, filtrado, limpieza y agrupación sobre un DataFrame real.
- **Practicamos:** visualización simple y exportación de resultados.

- **Próximamente:** POO avanzada y análisis más profundo con pandas.
- **Recomendación:**
 - Practicar imputación por modelos y validación cruzada.
 - Explorar visualizaciones interactivas.

¡Nos vemos en la próxima sesión!